



Short Exercise: Machine Learning

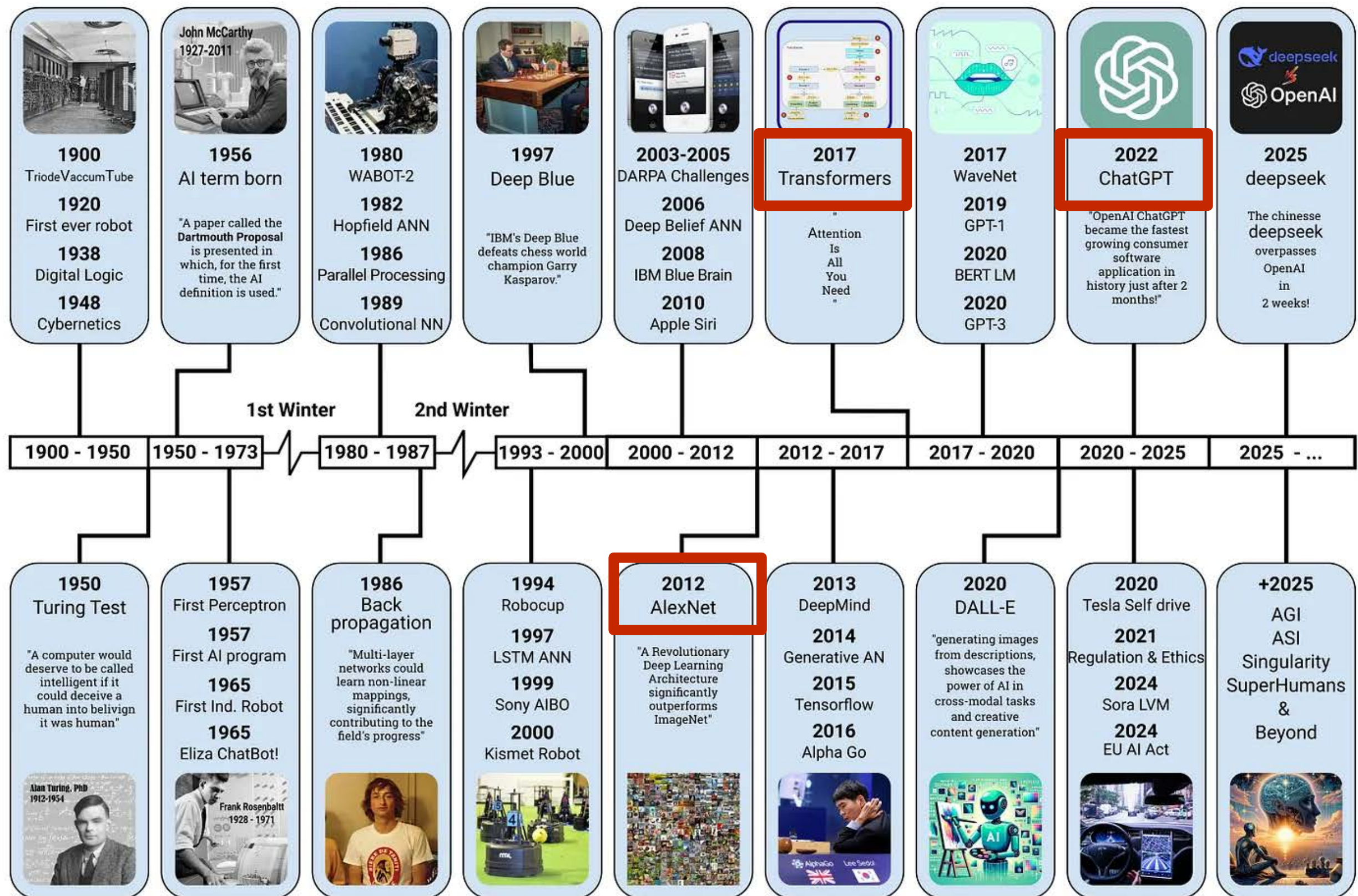
Introductory slides

CMSDAS 2026 Hefei @ USTC

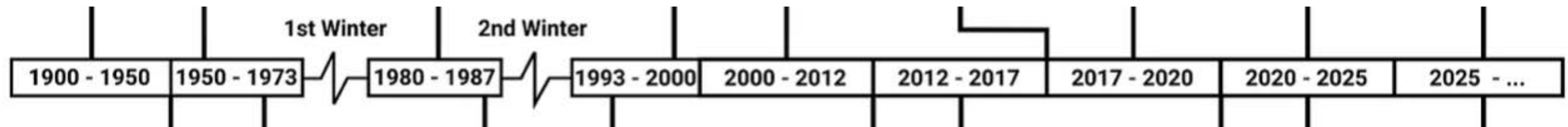
Instructor: Congqiao Li (Peking U)

24 March, 2026

Machine learning timeline...

Image [credit](#)

Machine learning × HEP

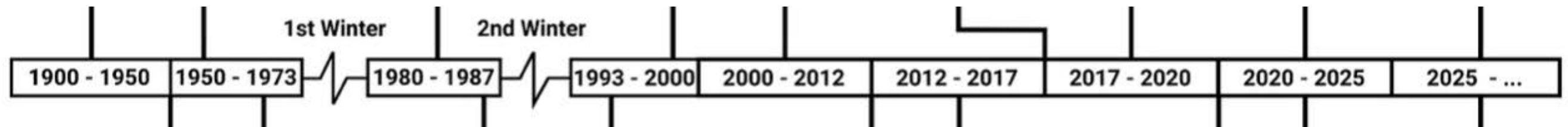


A rapidly evolving field: popular ML paradigms in HEP shift with time!

- ~2012 (Higgs discovery): **Decision Trees** dominate (ROOT TMVA); NNs (known as “artificial NNs”) not yet competitive
- 2012–2018: **Deep NNs** gain traction; **Convolutional NNs/recurrent NNs** introduced to HEP; BDTs upgraded to **XGBoost** (mainstream); popular use of generative models: **GANs/VAEs**
- 2018–2022: Rise of **GNNs**; growing focus on physics-inspired and symmetry-aware architectures
- 2022–present: **Transformers** as a unified framework; **generative models** (diffusion models) being matured



Machine learning × HEP



A rapidly evolving field: popular ML paradigms in HEP shift with time!

- ~2012 (Higgs discovery): **Decision Trees** dominate (ROOT TMVA); NNs (known as “artificial NNs”) not yet competitive
- 2012–2018: **Deep NNs** gain traction; **Convolutional NNs/recurrent NNs** introduced to HEP; BDTs upgraded to **XGBoost** (mainstream); popular use of generative models: **GANs/VAEs**
- 2018–2022: Rise of **GNNs**; growing focus on physics-inspired and symmetry-aware architectures
- 2022–present: **Transformers** as a unified framework; **generative models** (diffusion models) being matured



Exercise design philosophy:

stay close with current trends —
stronger focus on Transformer-
based use cases!

Outline

This tutorial includes three sessions

→ **Session 1:**

Introduction to BDTs and DNNs. You'll train a BDT with XGBoost and a simple DNN with PyTorch for a signal vs. background classification task, and explore how different parameters affect performance.

→ **Session 2:**

Using Transformers for classification with token-based inputs (jets/leptons as tokens). Focus on understanding the architecture and tuning key hyperparameters.

→ **Session 3:**

Going beyond classification with Transformers in particle physics. We look at incorporating physics-inspired features, comparing different inputs, and applying the model to a jet assignment problem.

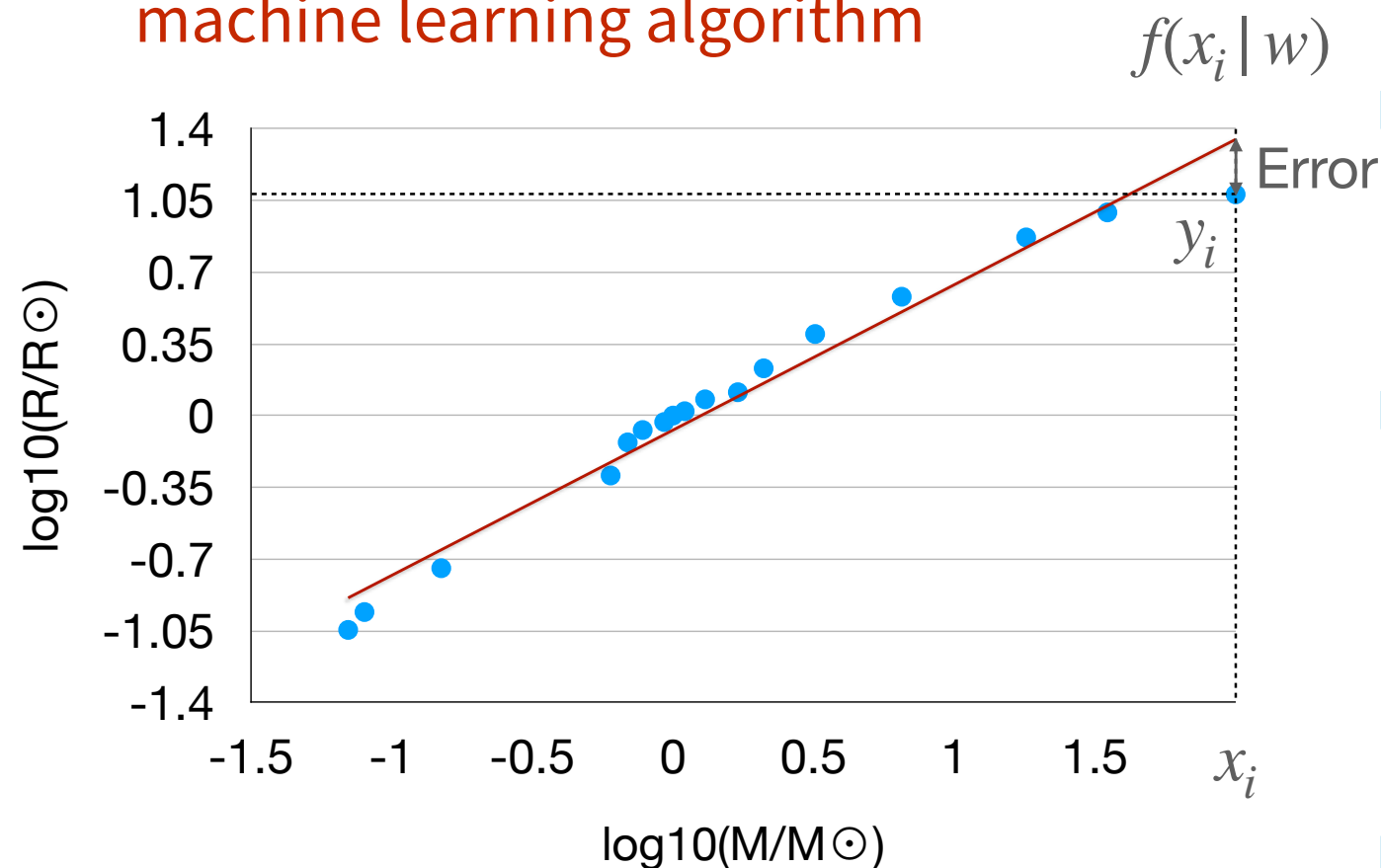
Brief overview on ML×HEP

*Several slides referring to early CMSDAS materials (2025 at [DESY](#) & [Fermilab](#))

Machine Learning?

Machine learning: “data-driven” algorithms that learn from sample data to make predictions or decisions, without explicitly encoding rules in the program

Example: **linear regression** is a machine learning algorithm



▶ **Linear model:**

$$f(x | w) = w^T x \quad (w \in \mathbb{R}^{D+1})$$

▶ **How do we select the parameters w ?**

▶ We want $y_i \approx f(x_i | w)$

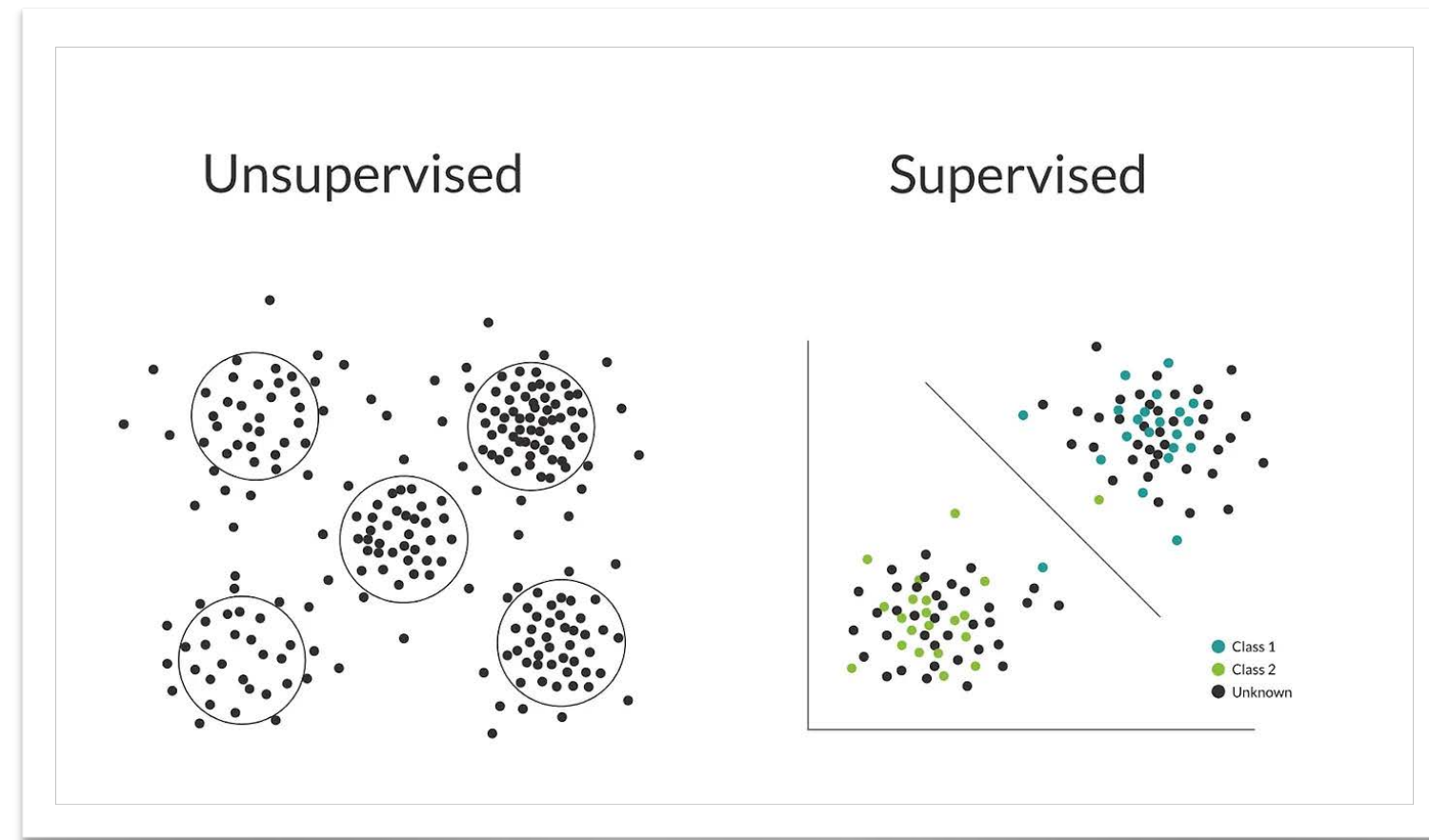
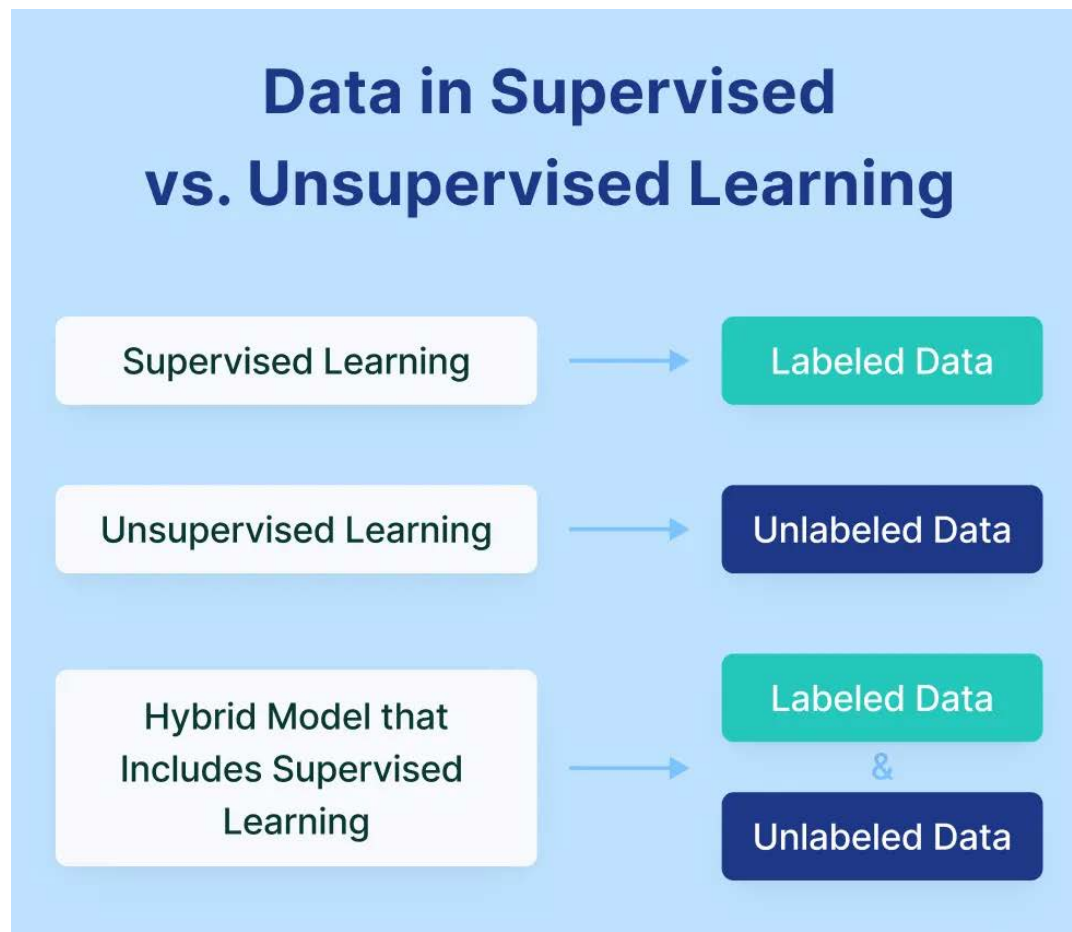
▶ **Squared loss:** $L(y, y') = (y - y')^2$

(Least squares)

Learning objective: $\arg \min_w \sum_{i=1}^N L(y_i, f(x_i | w)) = \arg \min_w \sum_{i=1}^N (y_i - w^T x_i)^2$

Supervised & unsupervised learning

Supervised vs. unsupervised learning:
Is labeled data used during training?



An intuitive understanding

Machine learning: general pipelines

- ▶ Training dataset: $S = \{(x_1, y_1), \dots, (x_N, y_N)\}$ where $x \in \mathbb{R}^D$ and $y \in \mathbb{R}$
- ▶ Model / hypothesis class: $f(x | w) = w^\top x$ (**linear models**)
- ▶ Loss function: $L(y, y') = (y - y')^2$ (**squared loss**)
- ▶ Optimization algorithm to minimize the learning objective:

$$\arg \min_w \sum_{i=1}^N L(y_i, f(x_i | w))$$

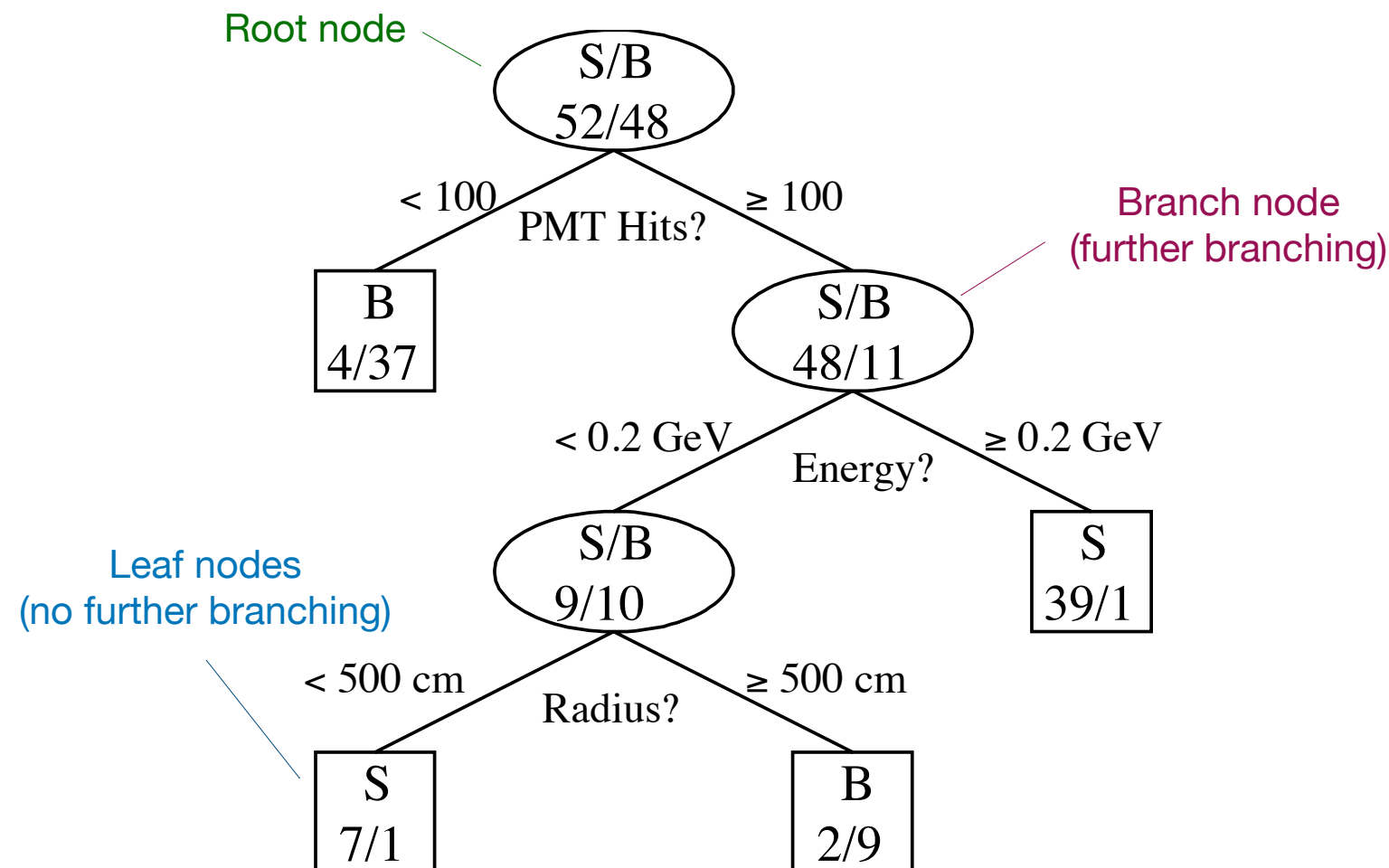
- ▶ Cross validation and model selection:



- ▶ Testing and deployment

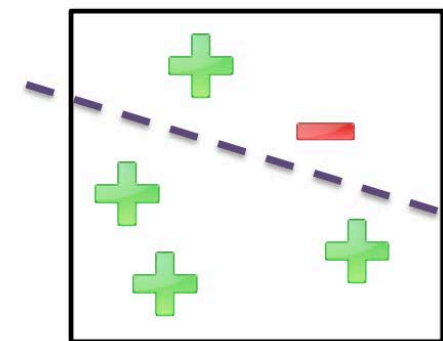
Important: if a testing set is available, never use it to make decisions on the model!

Decision trees

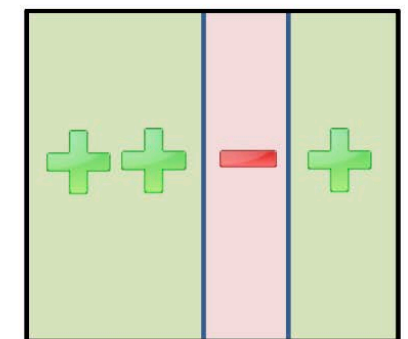
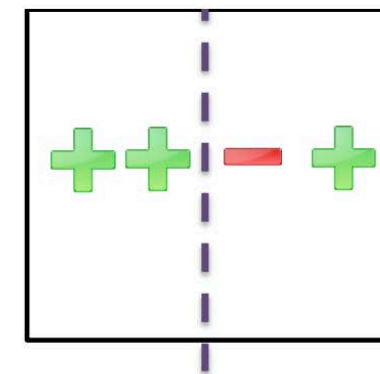
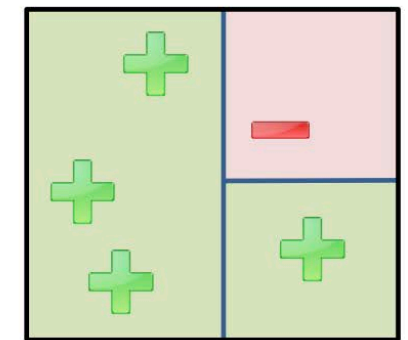


Decision-making using multiple variables

No linear model can achieve 0 error



Simple decision tree can achieve 0 error



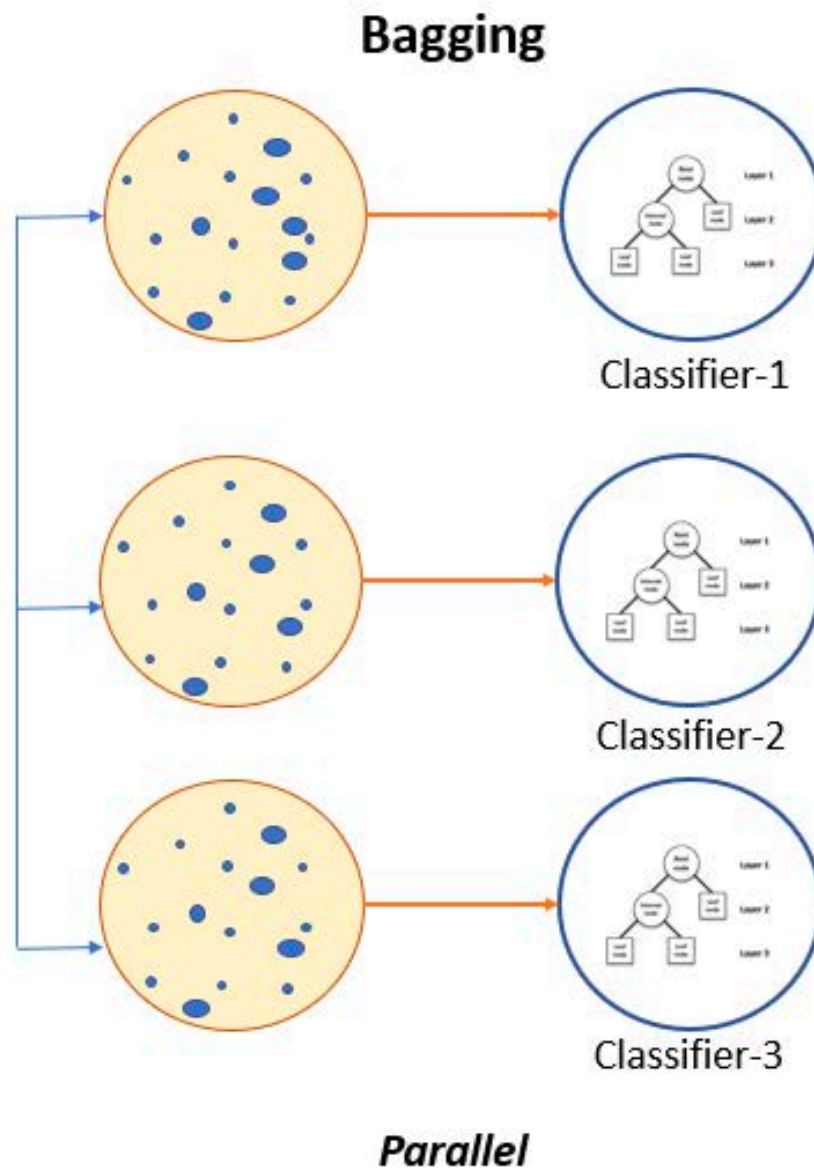
Decision tree: (1) a nonlinear model; (2) each individual decision uses a single variable

Decision trees

Improve performance using multiple decision trees: two historical approaches

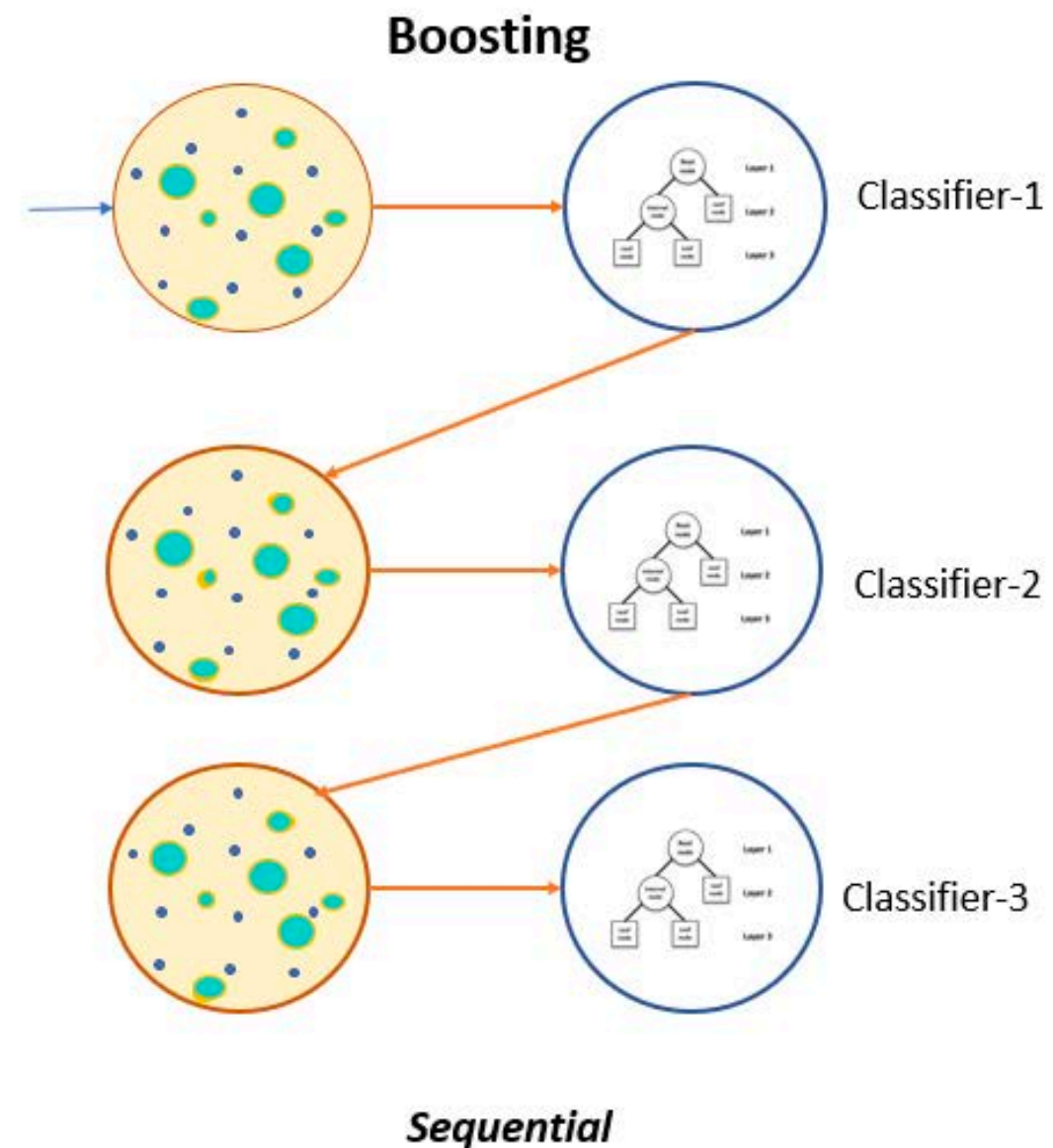
Bagging:

Train multiple models on different subsets of the data



Boosting: (commonly adopted)

- continuously fit new decision trees to the residuals of previous decisions
- the weights of the decision trees depend on their classification accuracy



Boosted decision tree (BDT)

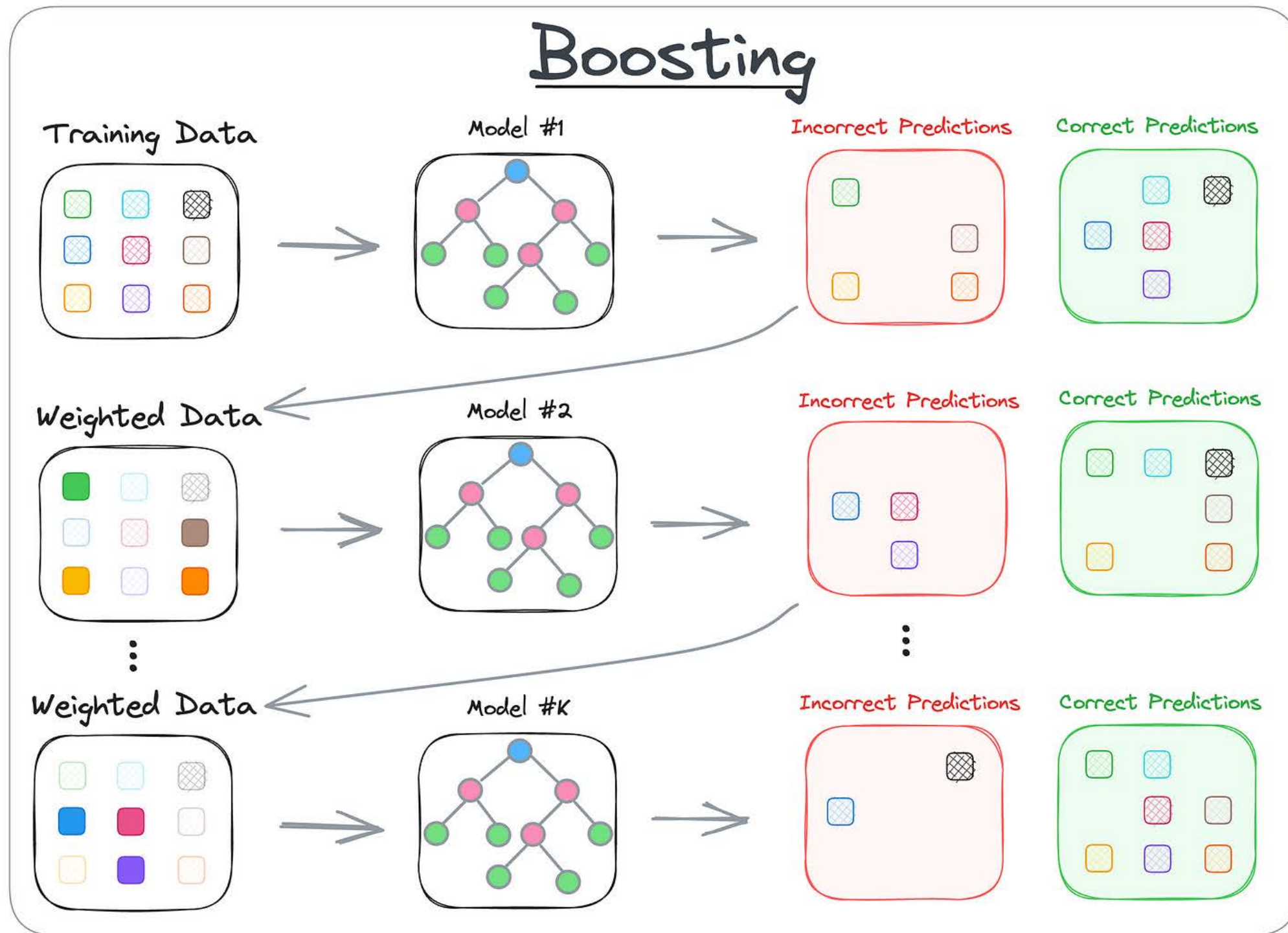
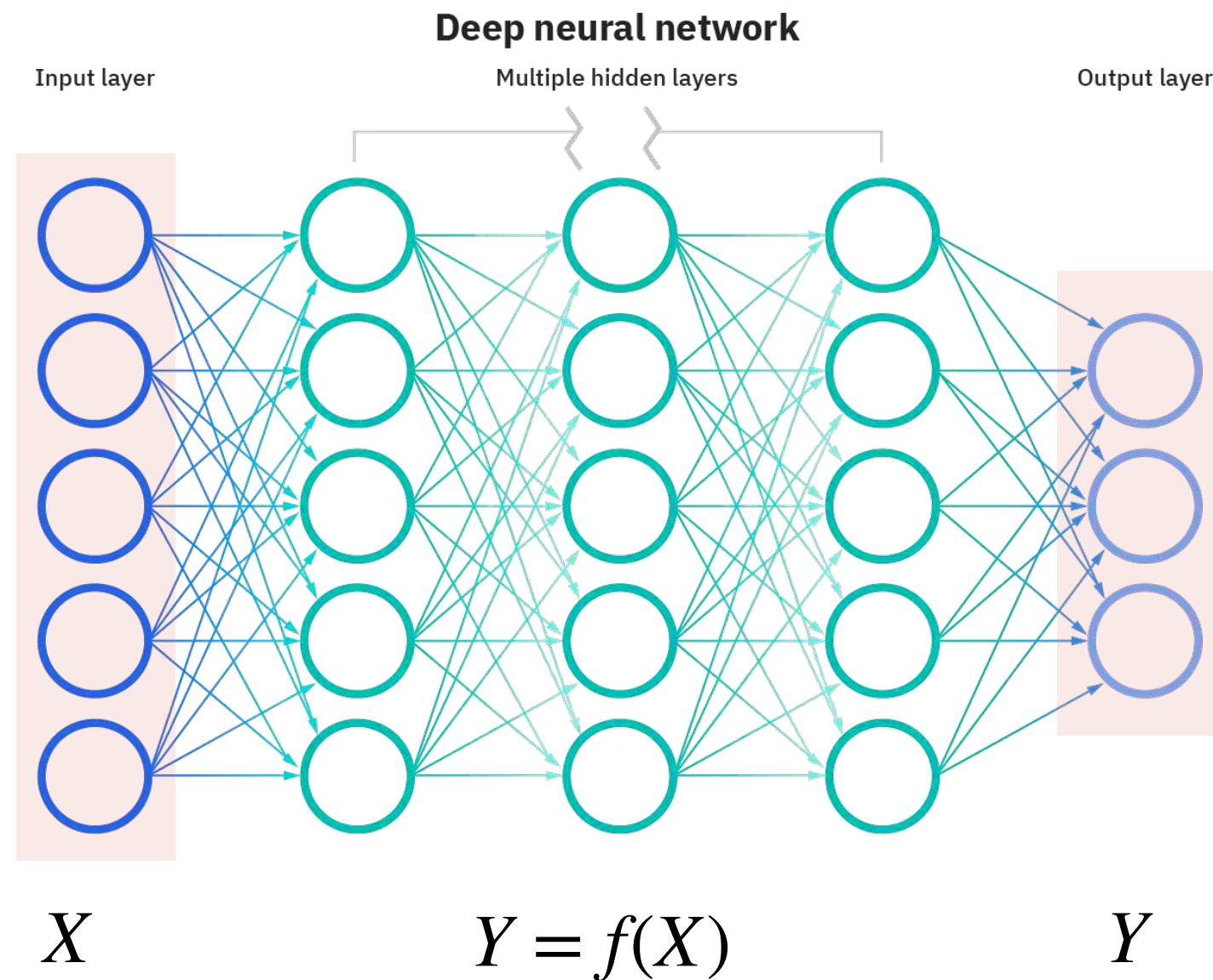


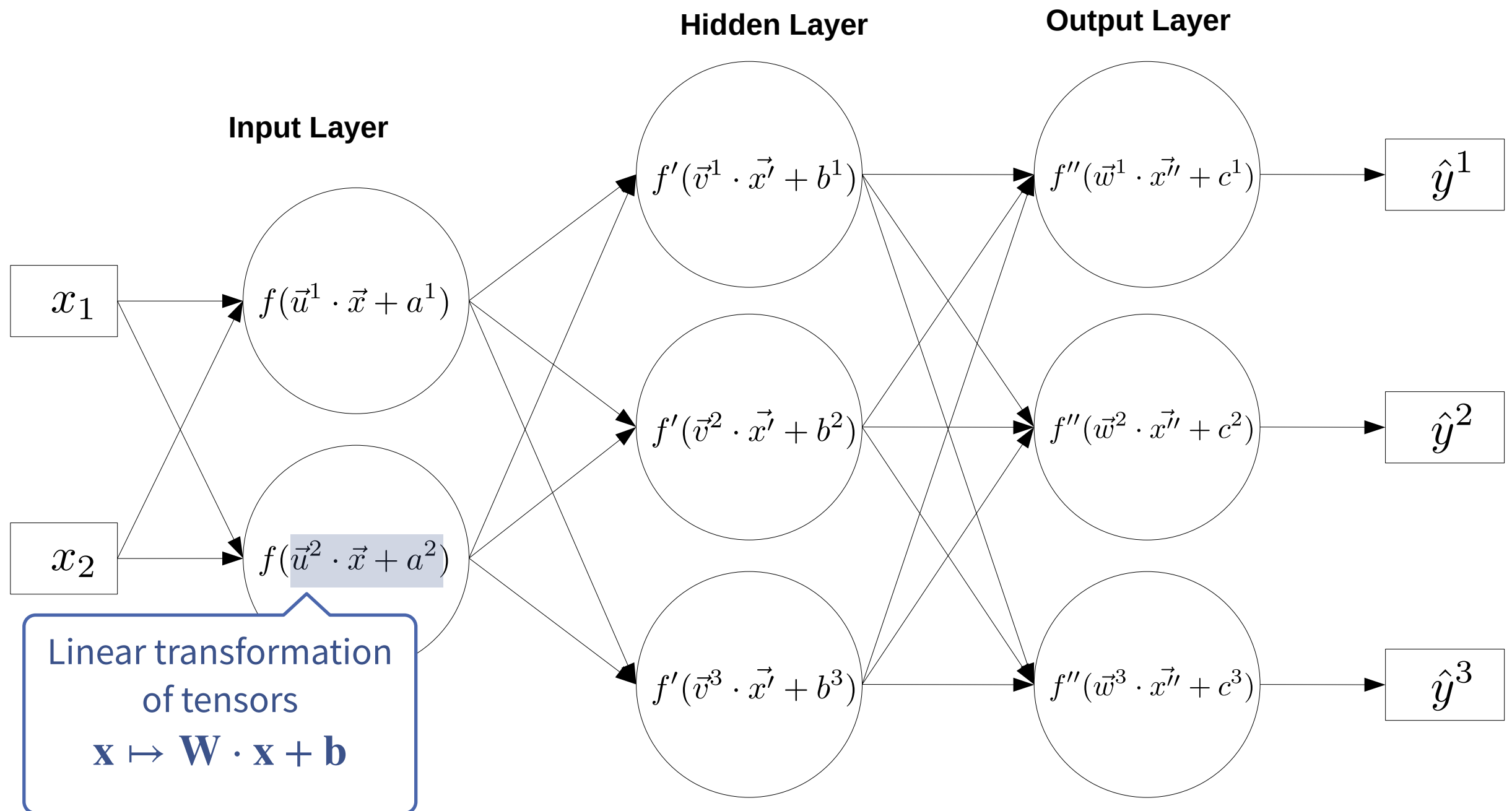
Image credit: [link](#)

Deep neural networks



Basic building block of DNN functions ($f(X)$):
Linear transformation + non-linear activation

Deep neural networks

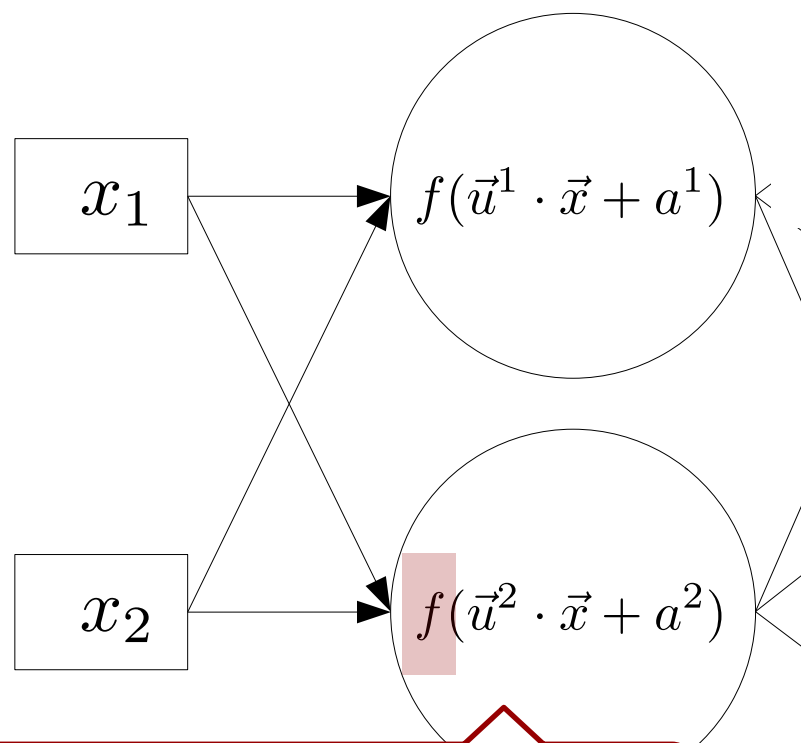


$$\begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}$$

Deep neural networks

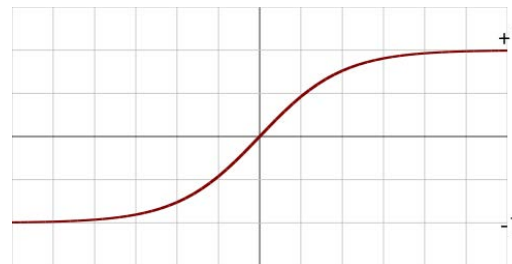
Several options on non-linear function

Input Layer



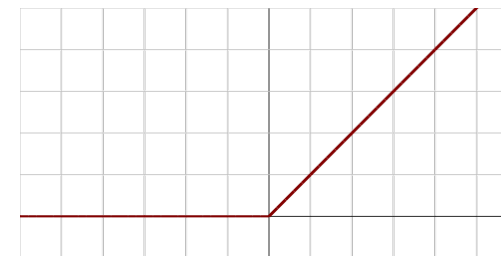
Non-linear activation

Hyperbolic tangent



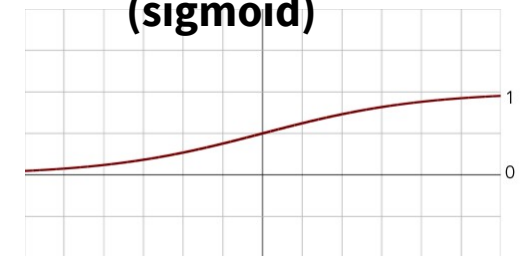
$$\hat{y} = \tanh x$$

Rectified Linear Unit (ReLU)



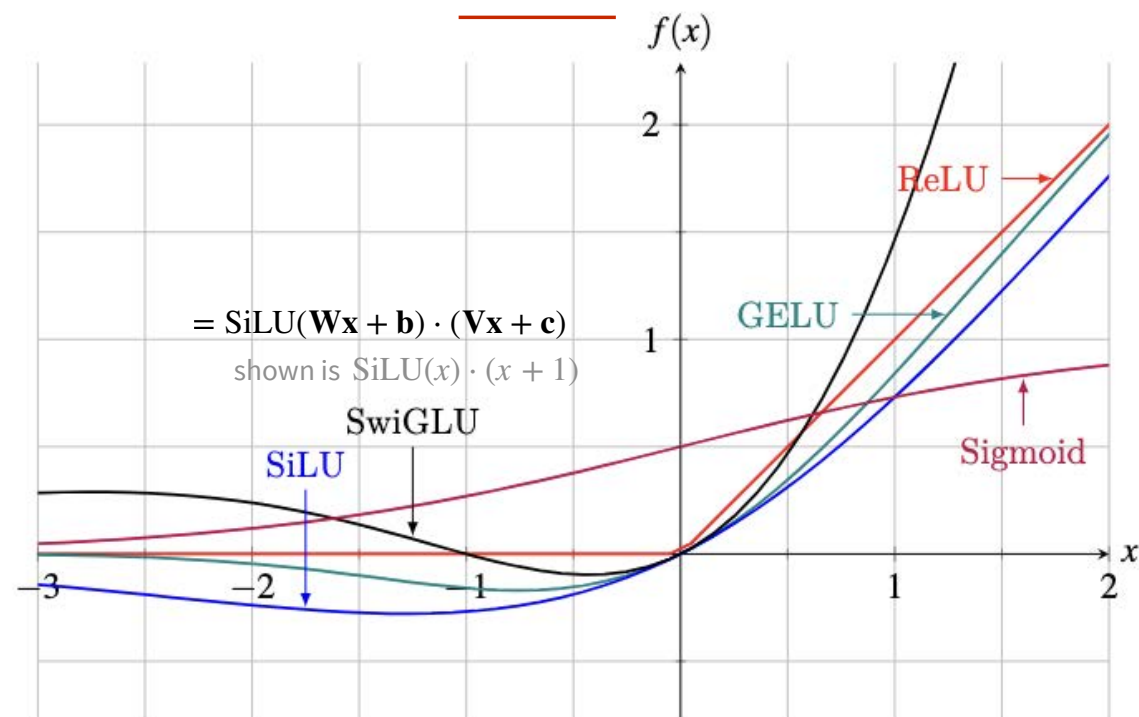
$$\hat{y} = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{otherwise} \end{cases}$$

Logistic function
(sigmoid)



$$\hat{y} = \frac{1}{1 + \exp(-x)}$$

Modern variants of ReLU

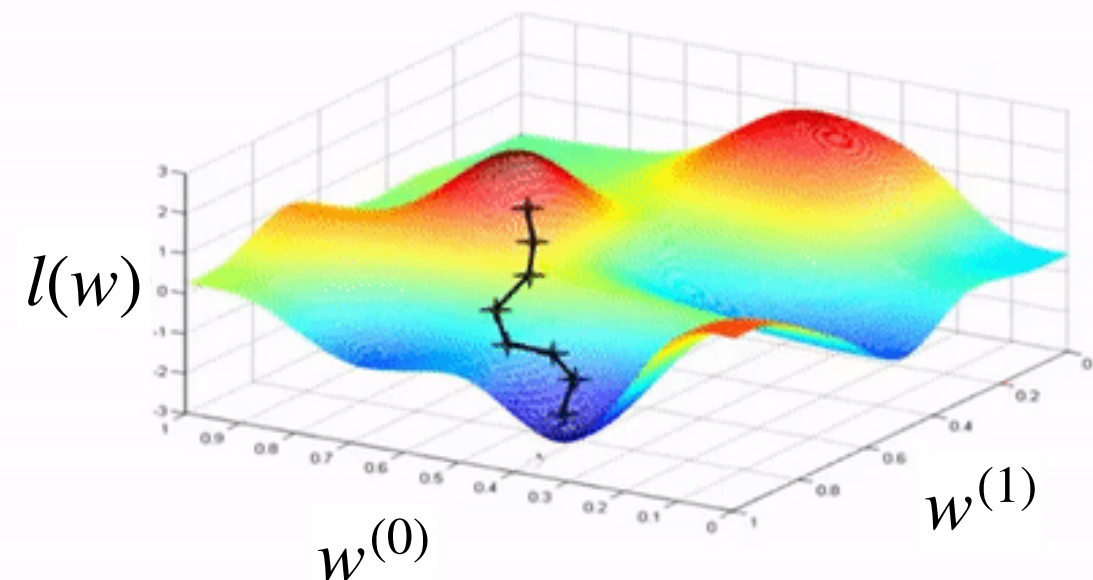


DNN optimization: gradient descent

- ▶ Set $w(t = 0)$ to some values (e.g., $w(0) = 0$ or some random value)
- ▶ At iteration t ,
 - ▶ Compute the **gradient** $\nabla_w l(w(t))$: direction of steepest increase of $l(w)$ at $w(t)$
 - ▶ Take a small step in the **opposite direction**:

$$w(t + 1) = w(t) - \eta \nabla_w l(w(t))$$

↑
Step size / learning rate



Loss functions

- **For classification:** Use cross-entropy

$$p_i = p(y_i = 1 | \mathbf{x}_i) = \sigma(h(\mathbf{x}_i))$$

$$L(\mathbf{w}, \mathbf{U}) = - \sum_i y_i \ln(p_i) + (1 - y_i) \ln(1 - p_i)$$

- **For regression:** Use squared error or something similar

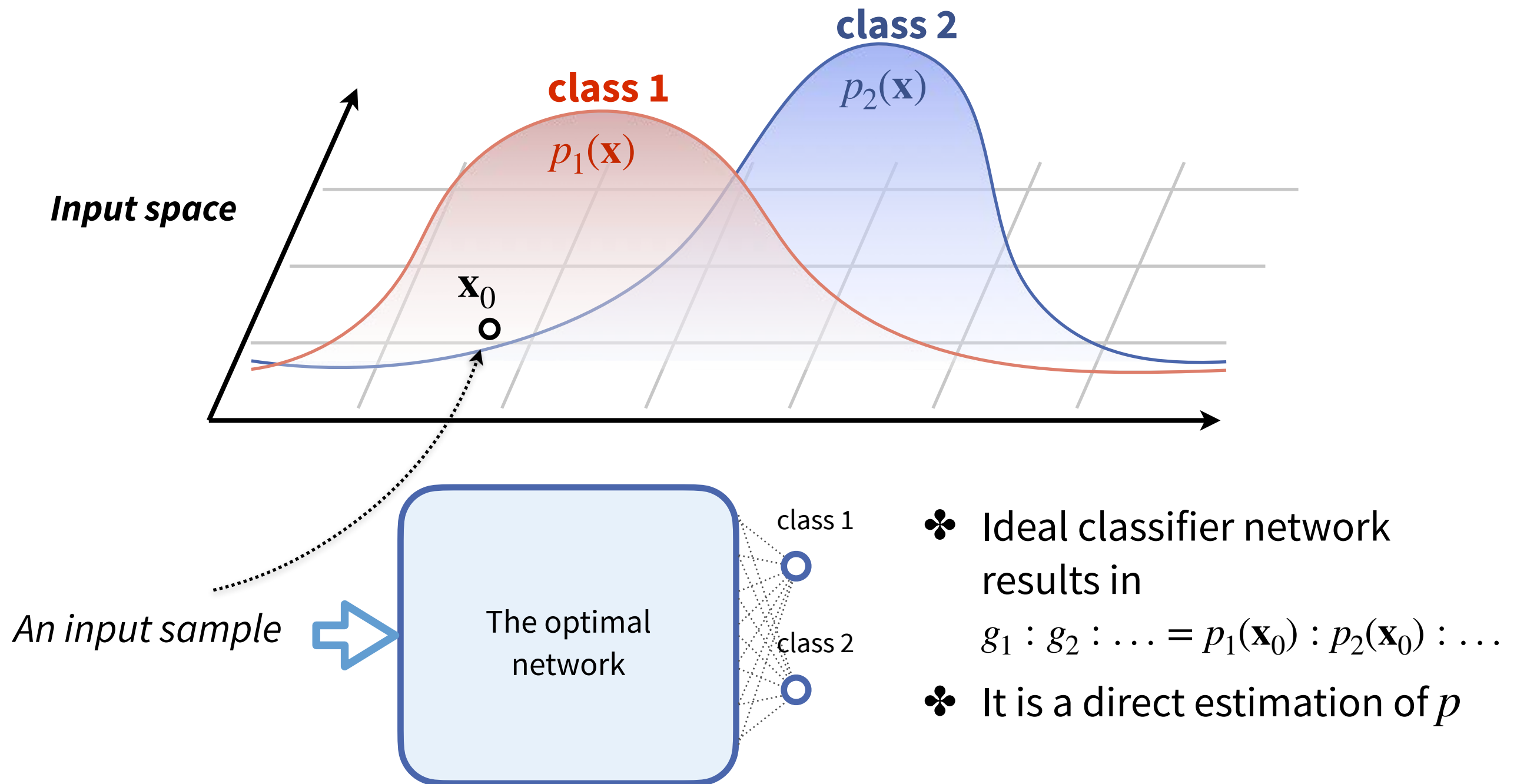
$$L(\mathbf{w}, \mathbf{U}) = \frac{1}{2} \sum_i (y_i - h(\mathbf{x}_i))^2$$

For example

- Target label: $y = 0$ (bkg)
network output “signal score” = 0.2
 $L = -\ln(0.8)$
- Target label $y = 1$ (signal)
network output “signal score” = 0.9
 $L = -\ln(0.9)$
- But why not use a simpler version??

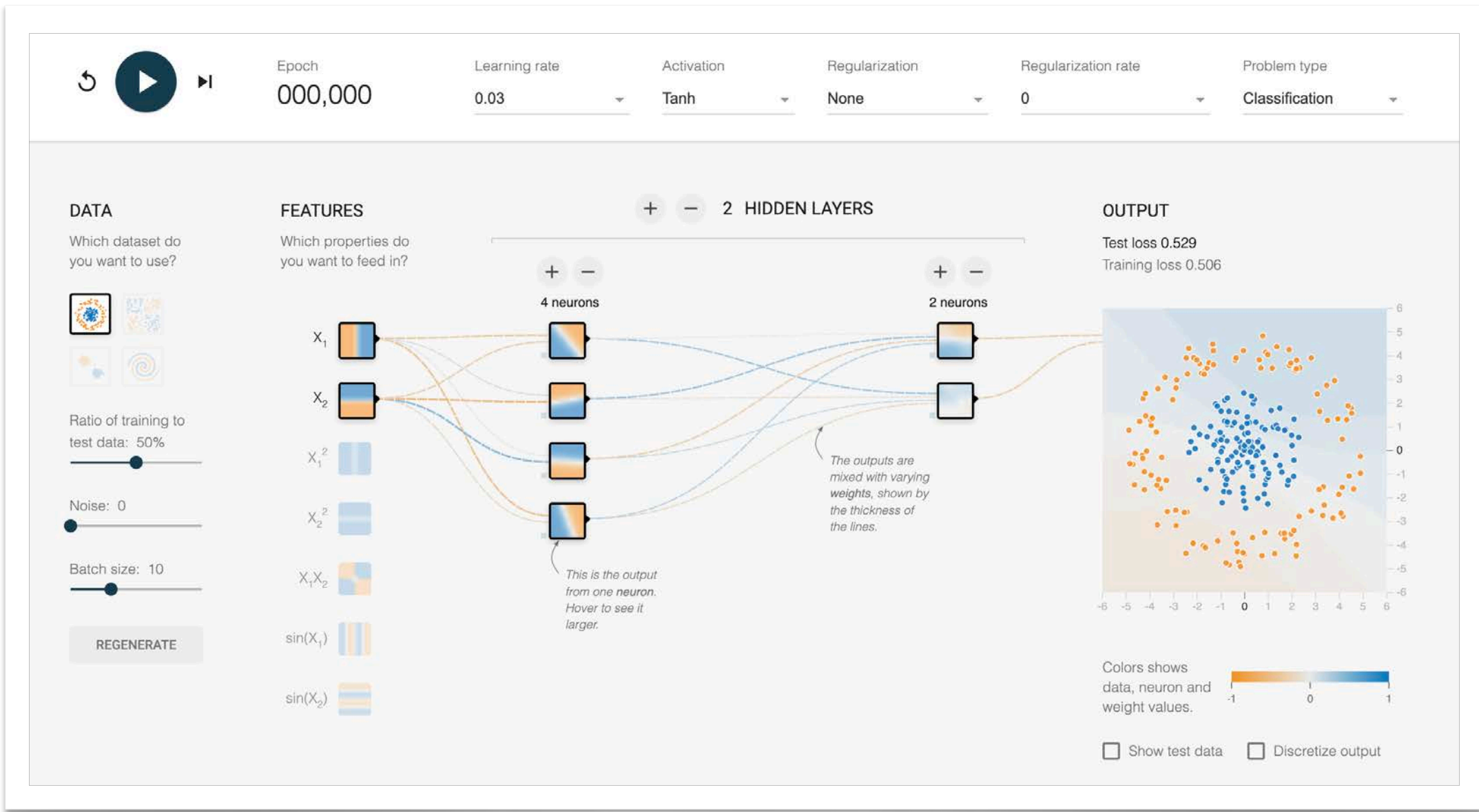
Intuition of cross-entropy loss

→ Optimize **cross-entropy** loss for classification tasks



DNN optimization

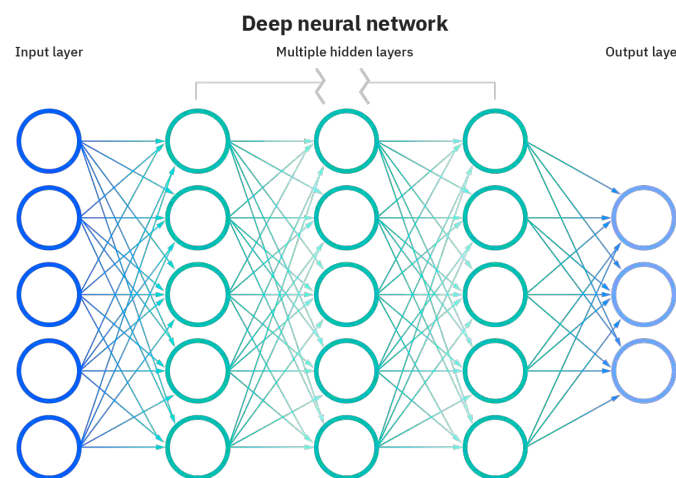
→ An intuitive example!



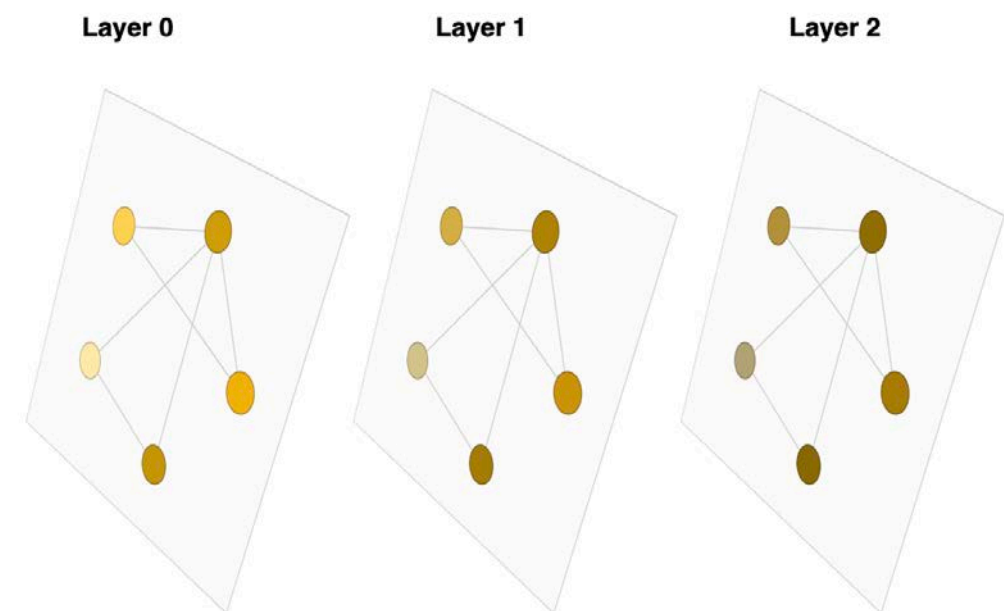
<https://playground.tensorflow.org/>

DNN optimization: input engineering

- To facilitate DNN optimization, data preprocessing is required:
 - ❖ Shift and scaling → e.g., transform input variables to have mean 0 and variance 1
 - ❖ Long-tailed distributions (e.g., jet p_T): using $\log(|x|)$, $\log(|x|+1)$ obtain distributions closer to Gaussian
- flattened 1D vector & tokenized input?



- When using DNN, we have to **flatten the input to 1D array**

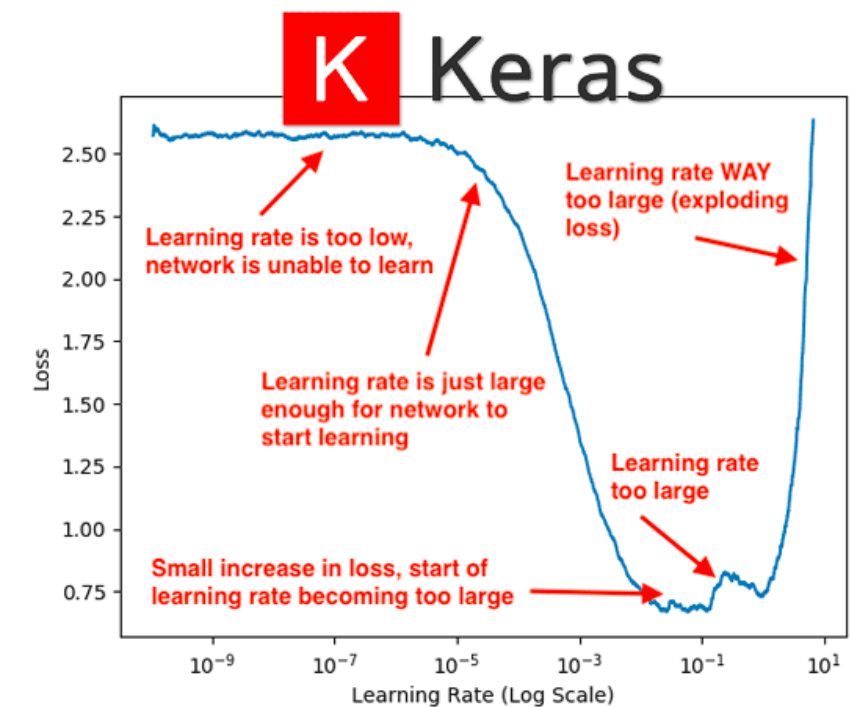


- Construct hierarchical inputs (**treat each lepton/jet or similar object as an input node/token**) → **preserve the intrinsic structure of the data**
- Allow as to use more expressive models, such as graph neural networks or Transformers for analysis

DNN optimization: hyperparameter tuning

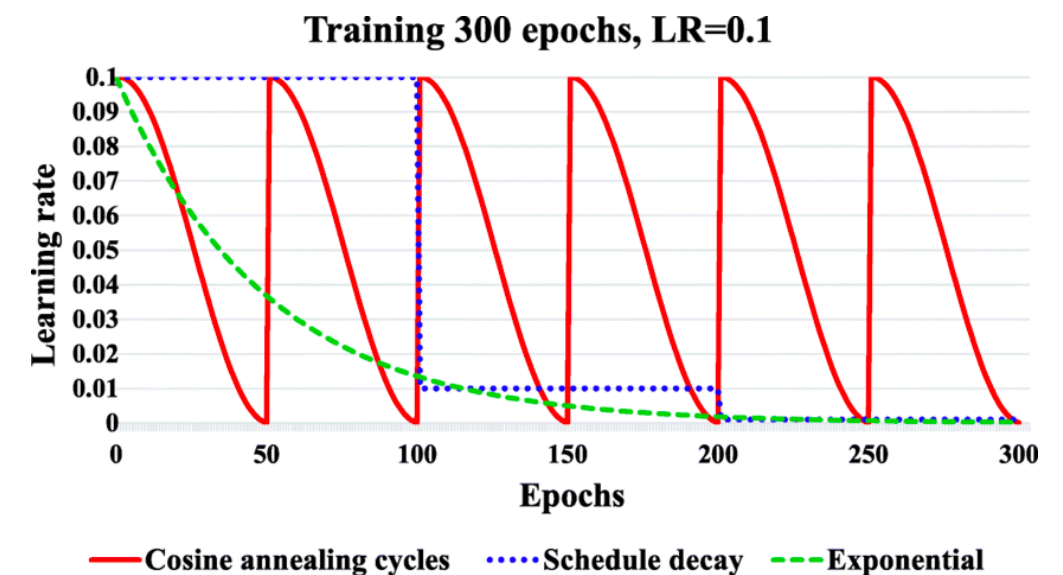
→ Learning rate

- ❖ The learning rate is the **most important hyperparameter**
- ❖ It determines the step size for updating parameters along the gradient at each training step; it can be tuned over orders of magnitude, and the optimal value is then selected



→ Learning rate decay

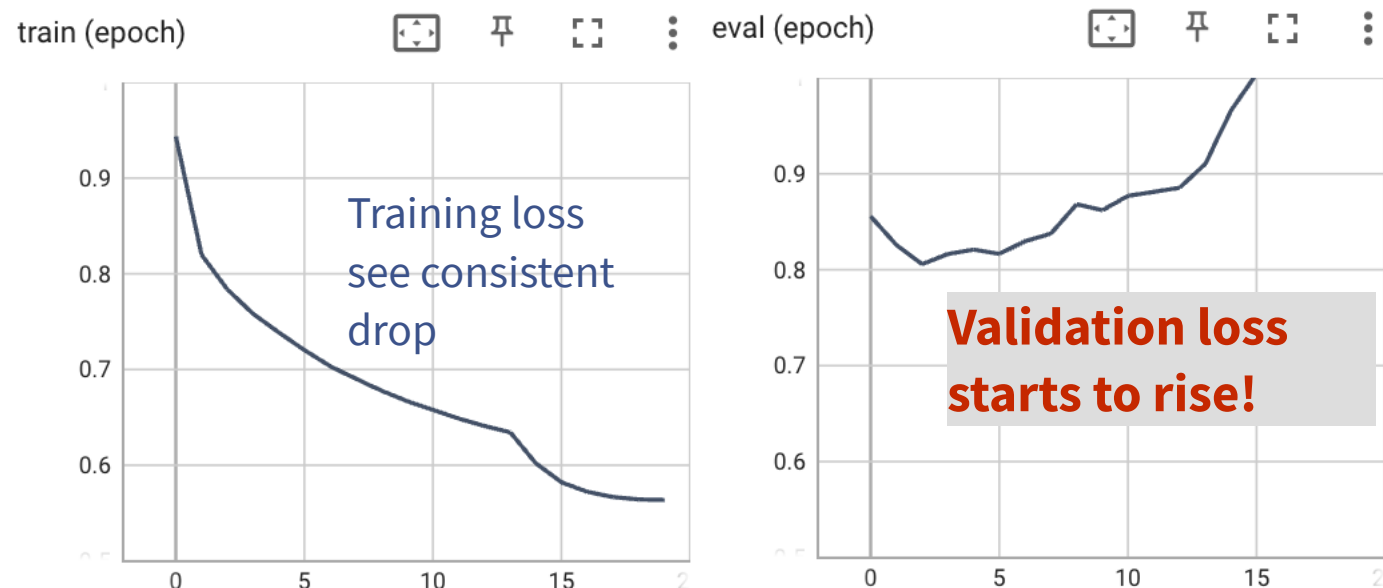
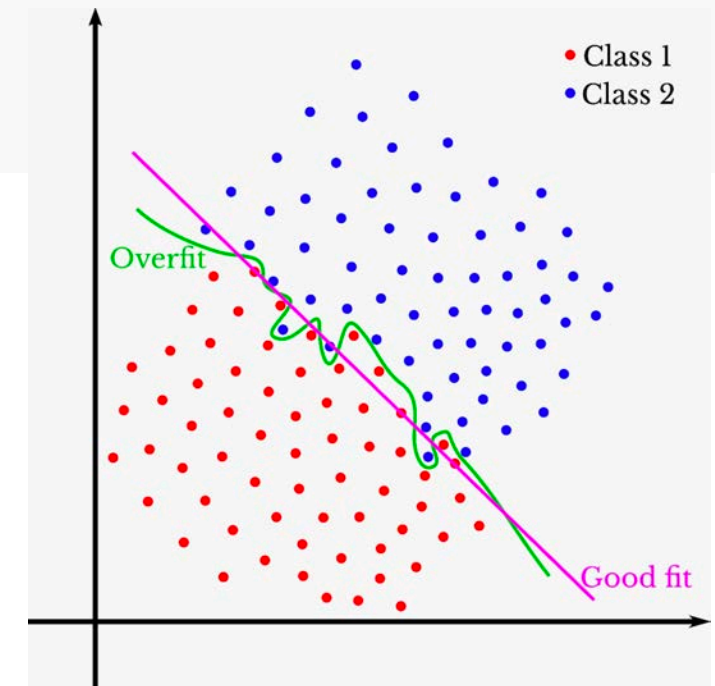
- ❖ What decay schedule to use? —options include step decay, exponential decay, and cosine annealing, each suited to different scenarios
- ❖ For basic usage, simple step decay or exponential decay is usually sufficient



DNN optimization: overfitting

→ Overfitting

- ❖ What we see: the loss on the training data keeps decreasing, but after each epoch, evaluation on the test (or validation) data shows that the loss no longer decreases and may even start to increase



→ Causes:

- ❖ model is too large, with many redundant parameters
- ❖ training dataset is too small to support deep neural network training (empirically, NNs generally require more data than BDTs)

→ Possible improvements:

- ❖ reduce model size (e.g., fewer layers or smaller layer widths)
- ❖ add dropout: a training technique that randomly deactivates a subset of neurons in each training batch

Let's begin our notebook sessions

Using SWAN/lxplus-gpu

Getting Started

Option 1: SWAN

You can open everything directly in SWAN. For environment setup, see `sesh0_intro.pdf`.

Open in  SWAN

Option 2: lxplus-gpu

1. Log in:

```
ssh <USER>@lxplus-gpu.cern.ch
```

2. On the lxplus-gpu node:

```
hostname # e.g. lxplus905.cern.ch

source /cvmfs/sft.cern.ch/lcg/views/LCG_109_cuda/x86_64-el9-gcc13-opt/setup.sh

git clone https://gitlab.cern.ch/das-hefei2026/exercise.git .
cd exercise/Short_exercise/Machine_Learning
jupyter lab --no-browser
```

You will see a link like `http://localhost:8888/lab?token=xxx`. Note the port number (e.g. 8888).

3. On your local machine:

```
ssh -L 8888:localhost:8888 <USER>@lxplus905.cern.ch # change 8888 and lxplus905 as provided above
```

4. Then open the link in your browser.

SWAN configuration

Configure your session

Generate URL

Software

Try out our new experimental interface based on JupyterLab and let us know your feedback!

User Interface [more...](#)

☒ Try the new JupyterLab interface (experimental)

Source [more...](#)

Software source: curated stack (LCG) or custom environment.

☒ LCG

☐ Custom Environment

Software stack [more...](#)

The software stack to associate to the container. See the [LCG package info](#) page.

Additionally, it is possible to use Python packages installed by the user on CERNBox. More information [here](#).

109 CUDA 12.5 (GPU)

☐ Use Python packages installed on CERNBox

Platform [more...](#)

AlmaLinux 9 (gcc13)

Environment script [more...](#)

e.g. \$CERNBOX_HOME/MySWAN/myscript.sh

You can click this for modern experience

Select a GPU node (important)

Session resources

CPU [more...](#)

4

Memory [more...](#)

16 GB

GPU [more...](#)

Tesla T4 (15 GB)

External computing resources

Spark cluster [more...](#)

None

HTCondor pool [more...](#)

None

Start new session

References

Resources within CMS

→ CMSDAS references

- ❖ A good summary list is presented [here](#)
- ❖ Additional suite of CMSDAS machine learning materials: <https://fnallpc.github.io/machine-learning-das/index.html>

→ CMS ML Group (<https://twiki.cern.ch/twiki/bin/viewauth/CMS/CMSMachineLearning>)

- ❖ Hosts the bi-weekly CMS ML Forum: <https://indico.cern.ch/category/12412/>
- ❖ Journal club: <https://cms-ml-journalclub.web.cern.ch/>
- ❖ Documentation: <https://cms-ml.github.io/>

→ CERN Inter-Experiment LHC Machine Learning Working Group (<https://iml.web.cern.ch/>)

- ❖ HEP ML Resources: <https://github.com/iml-wg/HEP-ML-Resources>
- ❖ HEP ML Living Review: <https://github.com/iml-wg/HEPML-LivingReview>